

# **Formal Methods and Specifications**

## **Introduction**

**Dr. Nadeem Akhtar**

Associate Professor

# Software Engineering

**–Software Engineering deals with the building of software systems that are so large or so complex that they are built by a team or teams of engineers**

- Multiple versions
- Are in service for many years
- Undergo changes during their lifetime to fix defects, to enhance existing features, to add new features, to remove old features, or to be adapted to run in a new environment

# Software Engineering

- [IEEE Std 610.12-1990's]

*The application of a **systematic, disciplined, quantifiable** approach to the development, operation, and maintenance of software*

- **Classical Engineering:**
  - Tools and mathematical training to specify the properties of the product separately from those of the design.

Example: An Electrical engineer relies on mathematical equations to verify that a design will not violate power requirements.

- **Software Engineering:**
  - **Mathematical tools are not well-developed**
  - **Relies more on experience and judgment rather than mathematical techniques.**

# Role of a Software Engineer

- Good programmer – data structures and algorithms – fluent in one or more programming languages.
- Familiar with several design approaches
- **Be able to translate vague requirements into precise specifications.**
- Able to converse with the user of a system in terms of application.

# Role of a Software Engineer

- Ability to move among **several levels of abstraction** at different stages of the project, from specific application procedures and requirements, to abstractions for the software system, to a specific design for the system and finally to the detailed coding level.

# **Role of a Software Engineer**

- **Develop skills to build a variety of models and to reason about those models.**
- **Different models are used in the requirement phase, in the design of the software architecture, and in the implementation phase.**



# Role of a Software Engineer

- Model might be used to answer questions about both the behavior of the system and its performance

# Software Life Cycle (1/6)

- **Requirement analysis and specification.**
  - After Feasibility study (precise costs and benefits of the software system)
  - Identify and document the exact requirements for the system.
  - Produce User manuals, plans for the system test.

***What the problem is***

# Software Life Cycle (2/6)

- **System design and specification**
    - ***Architectural or High-level design:*** defines the overall organization of the system in terms of high-level components and interactions among them.
    - ***Detailed design:*** components are decomposed into lower level modules with precisely defined interfaces.
- All design levels are documented in specification documents that keep track of the design decisions.

***How to solve the problem***

# Software Life Cycle (3/6)

- *The Requirements phase attempts to specify What the problem is*
- *The purpose of design phase is to specify a particular software system that will meet stated requirements*

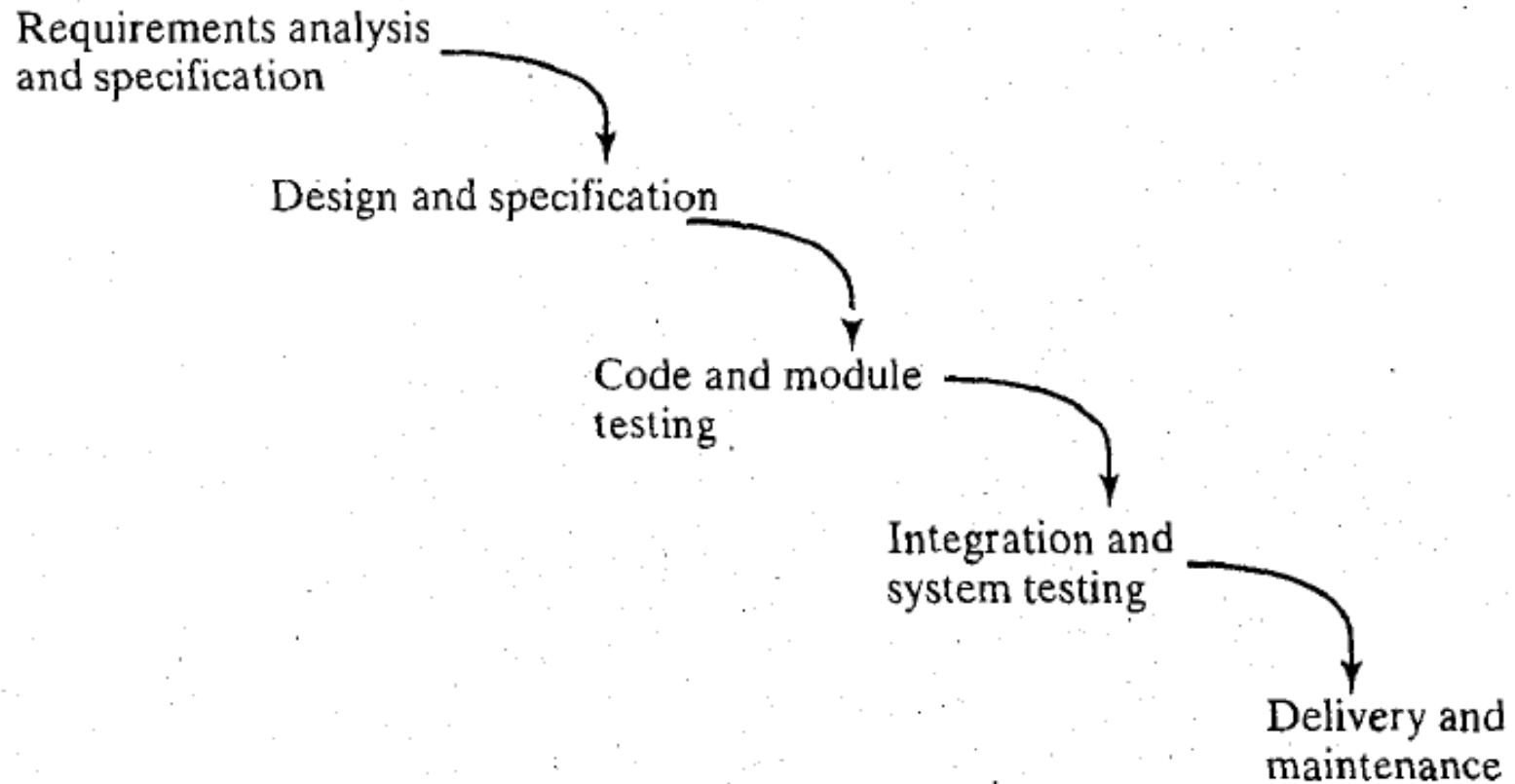
# Software Life Cycle (4/6)

- **Coding and module testing**
  - *Code that will be delivered to the customer as the running system.*
  - *Individual modules developed in coding phase are tested before delivered to the next phase*
  - *Other phases of the life cycle may also develop code, such as that for **prototypes, tests, and test drivers**.*

# Software Life Cycle (5/6)

- **Integration and system testing**
  - *Modules developed and tested individually are put together-integrated- in this phase and are tested as a whole system.*
- **Delivery and maintenance**
  - The system passes all the tests are delivered to the customer and enters the maintenance phase.

# Software Life Cycle (6/6)



# Correctness (1/5)

- Program provide functions specified in its **functional requirements specifications**
- A program is *functionally correct* if it behaves according to its stated functional specifications.



# Correctness (2/5)

- **Assumption:** Requirement specifications are available and it is possible to determine unambiguously whether a program meets the specifications.

# Correctness (3/5)

- *If requirement specification does exist, it is usually written in an informal style using natural language. Therefore it is most likely to contain many ambiguities.*
- *The Solution is to use formal methods to write mathematical based, concise and precise specifications which are well-defined.*

# Correctness (4/5)

- *Correctness is a mathematical property that establishes the equivalence between the software and its specification*
- *Correctness can be assessed more systematically and precisely depending upon how rigorous we are in specifying functional requirements.*

# Correctness (5/5)

- *Correctness of a program may be assessed through a variety of methods*
  - *Experimental approach e.g. testing*
  - *Analytic approach e.g. inspections or formal verification*
  - *Tools supporting extensive static analysis.*
  - *Using standard proven algorithms or libraries of standard modules.*

# Reliability (1/2)

- *Software is reliable if user can depend on it (Dependability).*
- *Reliability is the probability that the software will operate as expected over a specified time interval.*

# Reliability (2/2)

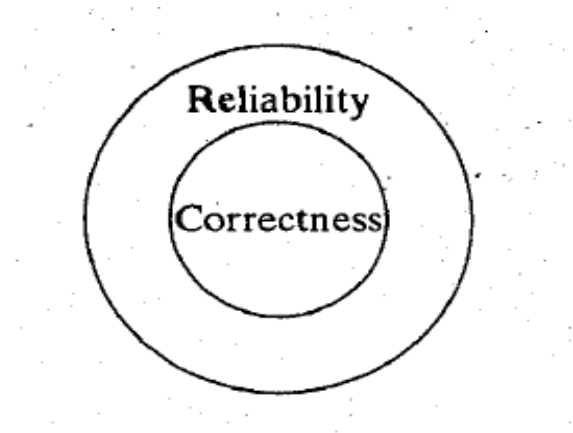
- *Correctness is a mathematical absolute quality: any deviation from its requirements makes a system incorrect. (deviation: minor or serious)*
- *Reliability is relative: If the consequence of a software error is not serious, the incorrect software may still be reliable.*

# Reliability and Correctness (1/2)

- **Assumption:** functional requirement specifications captures all the desirable properties of the application and that no undesirable properties are erroneously specified in it.
- The set of all reliable programs includes the set of correct programs, but not vice versa.

# Reliability and Correctness (2/2)

- Idealized situation requirements are assumed to be correct.
- Not all incorrect behaviors signify equally serious problems; that is, some incorrect behaviors may be tolerable.





# Robustness (1/5)

- A program is robust if it behaves “reasonably” even in circumstances that were not anticipated in the requirements specification

## Example

When a program encounters incorrect input data or some hardware malfunction (i.e. a disk crash).

# Robustness (2/5)

- **A program that assumes perfect input and generates an unrecoverable run-time error as soon as the user inadvertently types an incorrect command is not robust.**
- A program might be correct, though, if the requirements specification does not state what the program should do in the face of incorrect input.

# Robustness (3/5)

- Robustness is a difficult-to-define quality; after all, if we could state precisely what we should do to make an application robust, we would be able to specify its “reasonable” behavior completely. **Thus, robustness would become equivalent to correctness.**

# Robustness (4/5)

- **Example:**

Two bridges connecting two sides of the same river are both “correct” if they satisfy the stated requirements.

If however during an unexpected, unprecedented earthquake, one collapses and the other one does not, we can call the latter more robust than the former.

# Robustness (5/5)

Lesson learned from the collapse of the bridge will probably lead to more complete requirements for future bridges, establishing resistance to earthquakes as a correctness requirement.

# Robustness and Correctness

In conclusion, we can see that robustness and correctness are strongly related, without a sharp dividing line between them.

If we put a requirement in the specification, its accomplishment becomes an issue of correctness; if we leave the requirement out of the specification, it may become an issue of robustness. The border line between the two qualities is the specification of the system.

# Correctness, Robustness & Reliability

**In relation to the software production process.**

**A process is robust, for example, if it can accommodate unanticipated changes in the environment, such as new release of the operating system or the sudden transfer of half the employees to another location.**

# Correctness, Robustness & Reliability

**A process is reliable if it consistently leads to the production of high-quality products.**



# Performance (1/3)

**Efficiency** is an internal quality and refers to how economically the software utilizes the resources of the computer.

**Performance** on the other hand, is an external quality based on user requirements.

For example: A telephone switch may be required to be able to process 10,000 calls per hour.

***Efficiency affects, and often determines, the performance of a system***

# Performance (2/3)

**Performance is important because it affects the usability of the system.**

If a software system is too slow, it reduces the productivity of the users, possibly to the point of not meeting their needs.

If a software system uses too much disk space, it may be too expensive to run.

If a software system uses too much memory, it may affect the other applications that run on the same system (run slowly as OS tries to balance the memory usage of different applications).

# Performance (3/3)

**Performance is important because it affects the scalability of a software system.**

*An algorithm that is quadratic may work on small inputs, but not work at all on larger inputs.*

*A compiler that uses register allocation algorithm whose running time is the square of the number of program variables will run more and more slowly as the length of the program being compiled increases.*

# Techniques of Performance Evaluation

- *Analyze the Complexity of Algorithms used in the software.*
  - Average or Worst case behavior of algorithms – in terms of resource requirements e.g. time and space – in case of distributed systems in terms of number of messages exchanges.
- *Measurement Analyses*
  - Actual performance measurement by hardware or software monitors that collect data while the system is running.

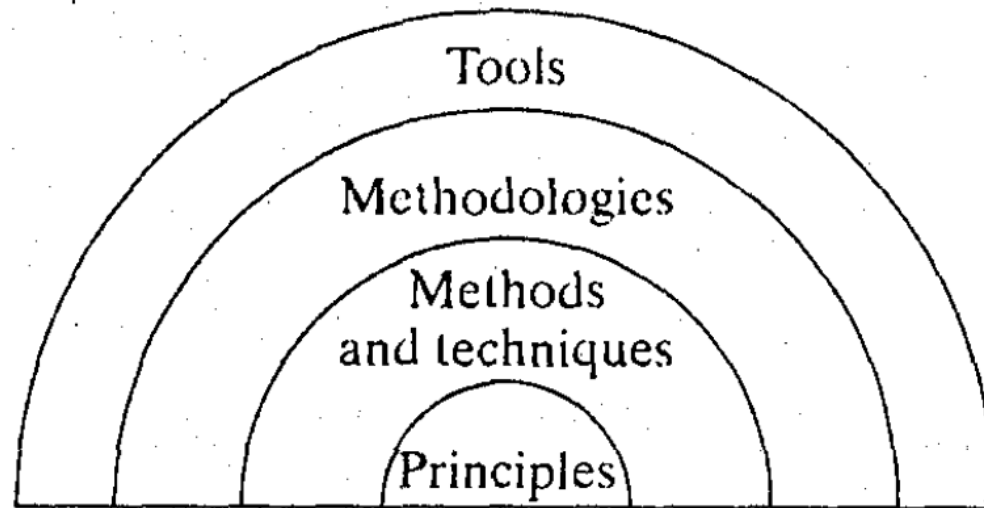
# Techniques of Performance Evaluation

- *Build a model of the product and analyze it.*
- *Build a model that simulates the product.*
  - **Analytical model** – easier to build but less accurate
  - **Simulation models** – costly to build but are more accurate.
  - Combine the two techniques – At start an analytic model to identify performance critical areas – simulation model of the performance critical areas

# Software Engineering Principles

- **Principles** are general enough to be applicable throughout the process of software construction and management.
- **Principles** are not sufficient alone to drive software development.
- **Principles** are general and abstract statements describing desirable properties of software processes and products.

## Relationships between principles, methods and techniques, methodologies, and tools



# Principles

- 1- Rigor and Formality**
- 2- Separation of Concerns**
- 3- Modularity**
- 4- Abstraction**
- 5- Anticipation of Change**
- 6- Generality**
- 7- Incrementalism**



# Methods and Techniques

- To apply principles, the software engineer should be equipped with **methods** and **techniques** that help incorporate the desired properties into processes and products.

# Methods and Techniques

- **Methods** are general guidelines that govern the execution of some activity; they are **rigorous, systematic, and disciplined** approaches.
- **Techniques** are more technical and mechanical than methods. They have more restricted applicability.

# Principles

# Rigor and Formality (1/8)

Rigor defined as **precision** and **exactness**.

*Rigorous approach* – that can repeatedly produce reliable products, control their costs, and increase our confidence in their reliability.

Highest degree of Rigor is **formality**.

# Rigor and Formality (2/8)

- Formality is stronger requirement than rigor. It requires the software process to be driven and evaluated by mathematical laws.
- Formality implies rigor, but the converse is not true. One can be rigorous and precise even in an informal setting.

# Rigor and Formality (3/8)

- Example

Engineer can rely on past experience and rules of thumb to design a small bridge, to be used temporarily to connect the two sides of a creek.

If the bridge is large and permanent one, engineer should use a mathematical model to verify whether the design is **safe**.

Engineer would use a more sophisticated mathematical model if the bridge is exceptionally long or if it is built in an area of seismic activity.

# Rigor and Formality (4/8)

Engineer (mathematician) must be able to identify and understand the level of rigor and formality that should be achieved depending on the difficulty and criticality of the task. The level may even vary for different parts of the same system.

# Rigor and Formality (5/8)

- **Example:**

*Critical parts* - such as the scheduler of a real-time operating systems kernel.

Security component of an electronic commerce system.

Must have a formal description of their intended functions and a formal approach to their assessment.

Well-understood and standard parts would require simpler approaches.



# Rigor and Formality (6/8)

- The description of what a program does may be given in a rigorous way by using natural language;
- It can also be given formally by providing a formal description in a language of logical statements.

# Rigor and Formality (7/8)

- The advantage of formality over rigor is that formality may be basis of mechanization of the process.

## For example:

One can use the formal description of the program to create the program (if the program does not exist yet) or to show that the program corresponds to the formal description (if the program and its formal specification exist)

# Rigor and Formality (8/8)

- Programming:

Programs are formal objects.

They are written in a language whose **syntax** and **semantics** are fully defined.

Programs are formal descriptions that may be automatically manipulated by compilers; They are checked for formal correctness.

# Separation of Concerns (1/8)

- Allows us to deal with different aspects of a problem, so that we can concentrate on each individually.
- It is a common-sense practice that we try to follow in our everyday life to overcome the difficulties we encounter.

# Separation of Concerns (2/8)

- *Software development – to master inherent complexity of software*
  - Decisions concern features of product; functions to offer, *expected reliability, efficiency with respect to space and time, user interfaces*
  - Decision concern the development process: *the development environment, the organization and structure of teams, scheduling, control procedures, design strategies, error recovery mechanisms.*
  - Decision concerns *economic and financial matters.*

# Separation of Concerns (3/8)

- Separate concerns in time

— For example:

University Professor i.e. Scheduling teaching-related activities such as holding classes, seminars, department meetings from 9 a.m. to 2 p.m. Engaging in research the rest of the time.

# Separation of Concerns (4/8)

## Example:

- Separate concerns in terms of *Qualities* of software that should be treated separately
- In case of software *Correctness* and *Efficiency* are dealt separately
  - *Correctness* is given priority (checked by formal correctness proofs and Complexity analysis) then restructure the program to improve its *Efficiency*.

# Separation of Concerns (5/8)

## Example:

- Separate concerns in terms of different *Views* of the software to be analyzed separately
- During requirement analysis of software
  - One view of **Flow of data** from one activity to another in a system.
  - Another view of **Flow of Control** that govern the way different activities are synchronized.



# Separation of Concerns (6/8)

## Example:

- Separate concerns in terms of *Size* allows to deal with *parts* of the same system separately
  - Separation by using *Modularity*

# Separation of Concerns (7/8)

## Example:

- Separate **Problem-domain** concerns from **Implementation-domain**.
- Problem-domain properties hold in general regardless of the implementation-domain.
  - **Employees Management System:** Problem-domain issues about employees in general
  - **Implementation-domain:** implementation of an employee as a data structure or object. Employee A reports to employee B. One object pointing to another.

# Separation of Concerns (8/8)

## Example:

- Separate **Managerial** and **Technical** issues in the software process.
- An analyst concentrating separately on **functional** and **non-functional** system requirements.

# Formalization Spectrum

Less Formal



← Natural language text description

← Mathematical proof

← Specification languages

← Specification + mathematical proof and/or  
automated tools

← Model checking, automated deduction/theorem  
proving

More formal